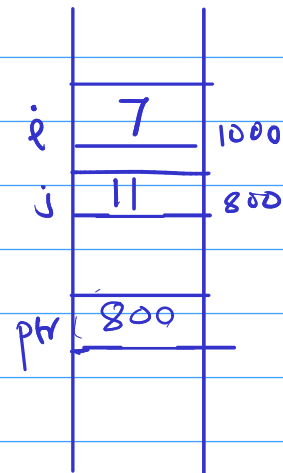


★ Revision

✓ int i=3, j=5;
 ✗ float *f = &i;
 ✓ int *ptr = &i;
 ✓ *ptr = 7; // i changes to 7.
 ✗ ptr = 7;
 ✗ *ptr = &j;



✓ ptr = &j; // changes value of ptr (which address of some int).
 ✓ *ptr = 11; // j changes to 11

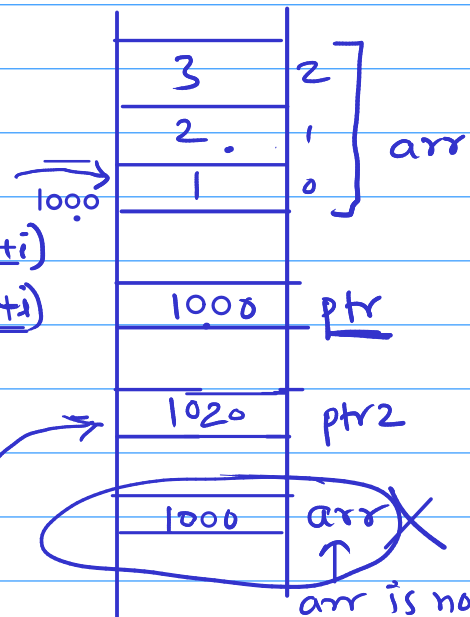
like const pointer to first element of array

✓ int arr[5] = {1, 2, 3};
 ✓ int *ptr = arr, *ptr2 = arr;
 for (int i = 0; i < 5; i++) {

Array indexing → printf("%d", arr[i]); //*(arr+i)
 printf("%d", ptr[i]); //*(ptr+i)

pointer offset → printf("%d", *(arr+i));
 printf("%d", *(ptr+i));

array traversal | printf("%d", *ptr2);
 ptr2++;



Pointer incrementation

ptr = ptr + 3

{
 *(arr+i)
 arr[i] } same

✗ *ptr[i] // ✗ *(*ptr+i)
 int i;
 *i ✗

★ Objectives of today's lecture

- Understand pointer arithmetic
- function and pointer.

⇒ Pointer arithmetic

✓ `int arr[10] = {11, 12, 13, 14, 15};`

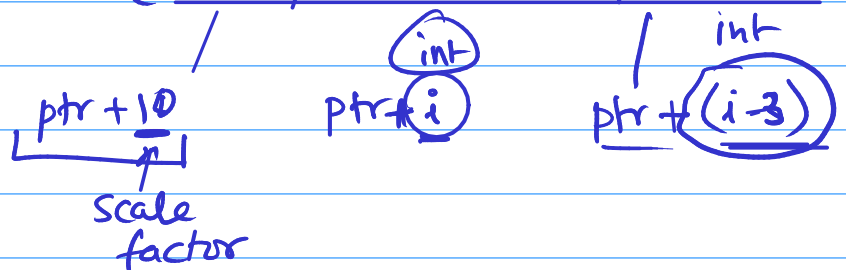
✓ `int *ptr = arr + 3;`

✓ `int *ptr2 = &arr[3];`

✓ `int *ptr3 = arr;`

`printf("%d %d %d", *ptr, *ptr2, *ptr3);`
 // 14 14 11

→ we can add int (const/variable/expression) to a pointer.



`double x = 1.1;`
`double *ptr = &x;`
`ptr = (ptr) + 2;`
 // Assume size of double to be 8 bytes

`int i;`
`int *ptr = &i;`
`ptr = ptr + 3.1;` ✗
`ptr = ptr - 6;` ✓
`ptr++;` ✓
`ptr -= 7;` ✓

`int i, j, k;`
`int *ptr1, *ptr2, *ptr3;`
`✗ *ptr1 = i;` // Uninitialize ptr1
`✓ ptr1 = &i;`
`ptr2 = &j;`
`ptr3 = &k;`
`✗ ptr3 = ptr1 + ptr2;`
 /
`3 + ptr2`
`ptr1 + 6` } valid

→ Two pointers can not be added.

ptr2 - ptr3 // valid, but meaningless.

⁽¹⁰⁰⁾
int arr[10];
int *ptr1 = arr, *ptr2 = &arr[7];
⁽¹⁰⁰⁾ ⁽¹²⁸⁾

int cnt = ptr1 - ptr2;

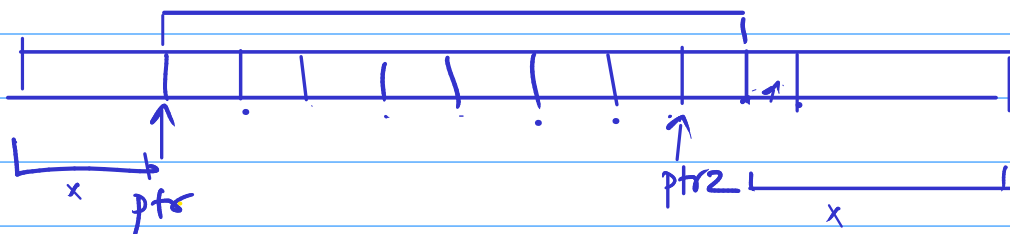
~~X~~ ²⁸ ⁽⁻⁷⁾
~~X~~ ⁻²⁸ ✓

↑ Addⁿ difference is divided
by scale factor

→ Two pointers of same type can be subtracted. But only meaningful if both pointers are pointing to elements of same array.

→ *, /, % X

→ >, <, >=, <=, ==, != ✓
for same type



```
for( ; ptr2 >= ptr; ptr++)  
    printf("%d", *ptr);
```

✓ *ptr1 + *ptr2
X ptr1 + ptr2

⇒ Pointer & function

Classic

Example — Swap values of two variables using function

pass by
value

X

```
void swap(int i, int j) {
    int temp = i;
    i = j;
    j = temp;
}
```

}

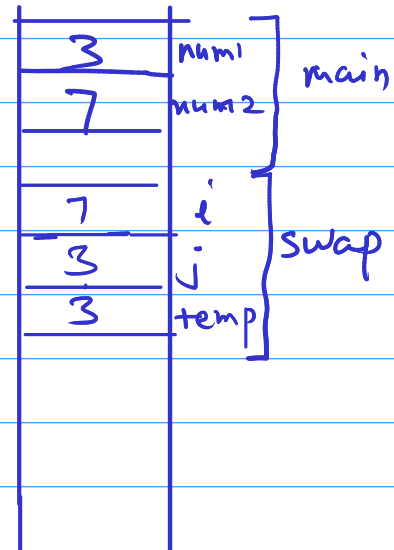
```
int main() {
```

```
    int num1 = 3, num2 = 7;
```

```
    → swap(num1, num2); // No swapping for num1 & num2
    printf("%d %d", num1, num2); // 3 7
```

```
    return 0;
```

}



```
void swap(int *i, int *j) {
    int temp = *i;
    *i = *j;
    *j = temp;
}
```

}

```
int main() {
```

```
    int num1 = 3, num2 = 7;
```

```
    → swap(&num1, &num2); // swapping num1 & num2
```

```
    → printf("%d %d", num1, num2); // 7 3
```

```
    return 0;
```

}

Pass
by
address

